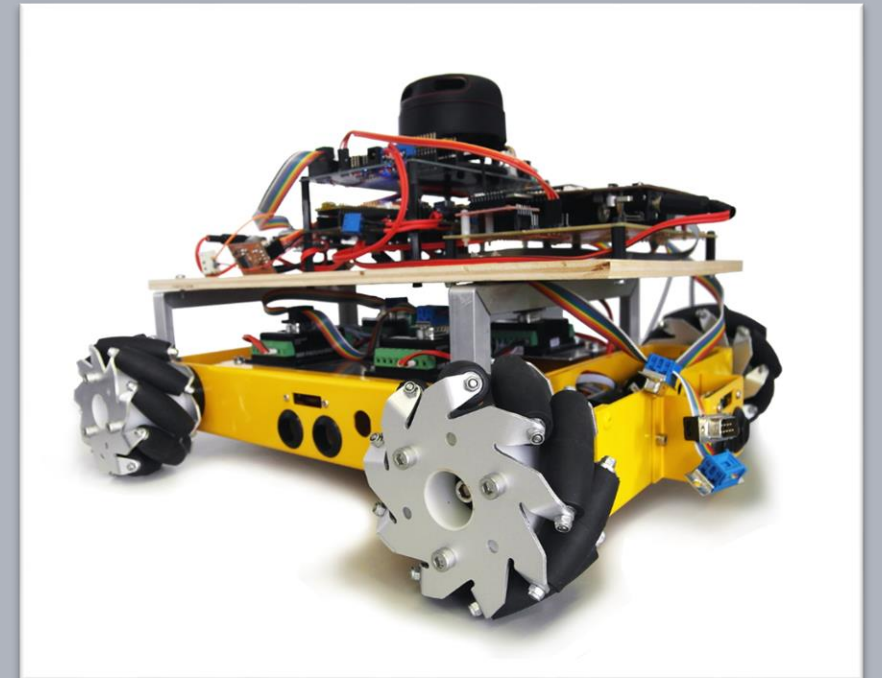


Advanced Embedded Architectures and Applications

Realtime and Worst Case Execution Time

Prof. Dr.-Ing. Peter Fromm



Content

- What means Realtime
- Timing issues
- Example Anticollision StudentCar
- Modelling the time behavior
- Cyclic Trigger Implementation
- Worst Case Execution Time Measurement



Realtime Criteria

“A realtime system is a system that must satisfy explicit (bounded) response-time constraints or risk severe consequences, including failure.

A failed system is a system which cannot satisfy one or more of the requirements laid out in the formal system specification.”

Laplane, 1993

“Realtime operation is the operation of a computing system, in such a way, that it is ensured, that the programs processing data are permanently operational ensuring that the requested results are available in a defined timeframe.

The processed data may come at predefined or random times”

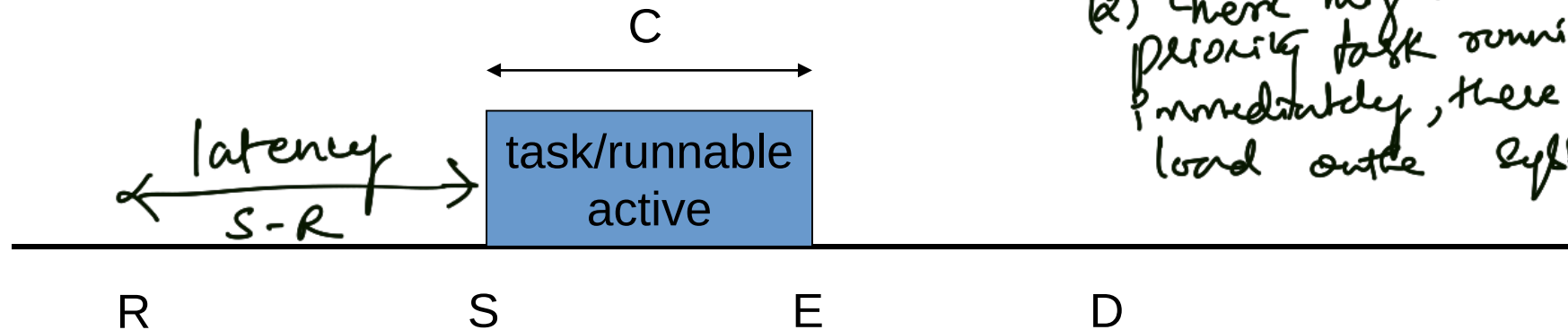
DIN 44300

Realtime constraints

Considering the definition of realtime - what is important?

- The time an activity starts?
- The time the activity takes?
- The time the activity is finished?

The OS assigns time to task/
runnable at time S and
early start time at R. There's
delay because
(1) The scheduler needs time for
? to own.
(2) There might be some higher
priority task running. If scheduled
immediately, there will be high
load on the system.



R: Release time – earliest time to start
D: Deadline – latest time to finish
S: Start time – real start time
E: Completion Time – real end time
C: Computation Time – runtime of the active task

Timing gates

As long as *release time* $\rightarrow$ *earliest start time*.

- $S > R$ (process starts after the release time)
- $E < D$ (process finished before the deadline)

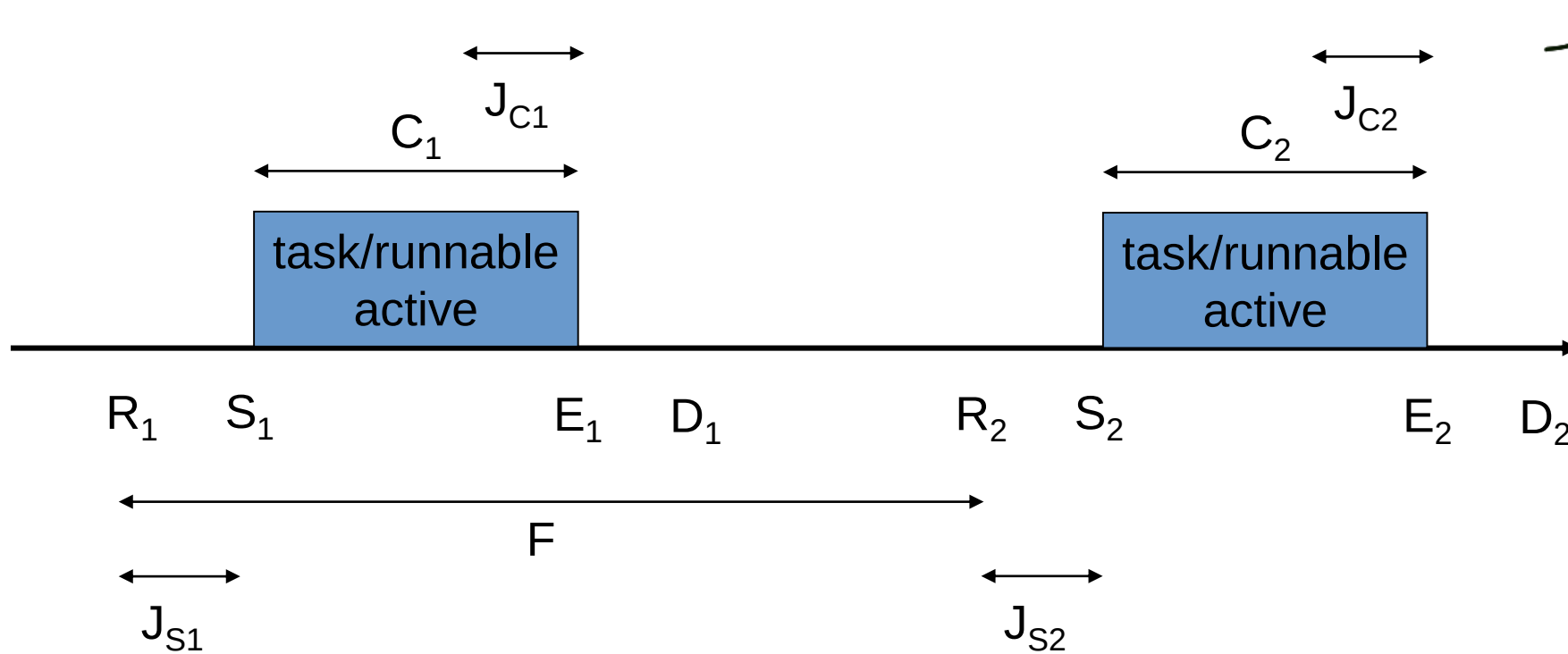
End time $\rightarrow$ *task time to finish task*

the realtime constraints for the task are met!

- **Latency** describes the delay between a stimulation and a response (S-R)
- **Jitter** is the deviation from true periodicity of a presumably periodic signal

Unfortunately reality is a bit more complex.

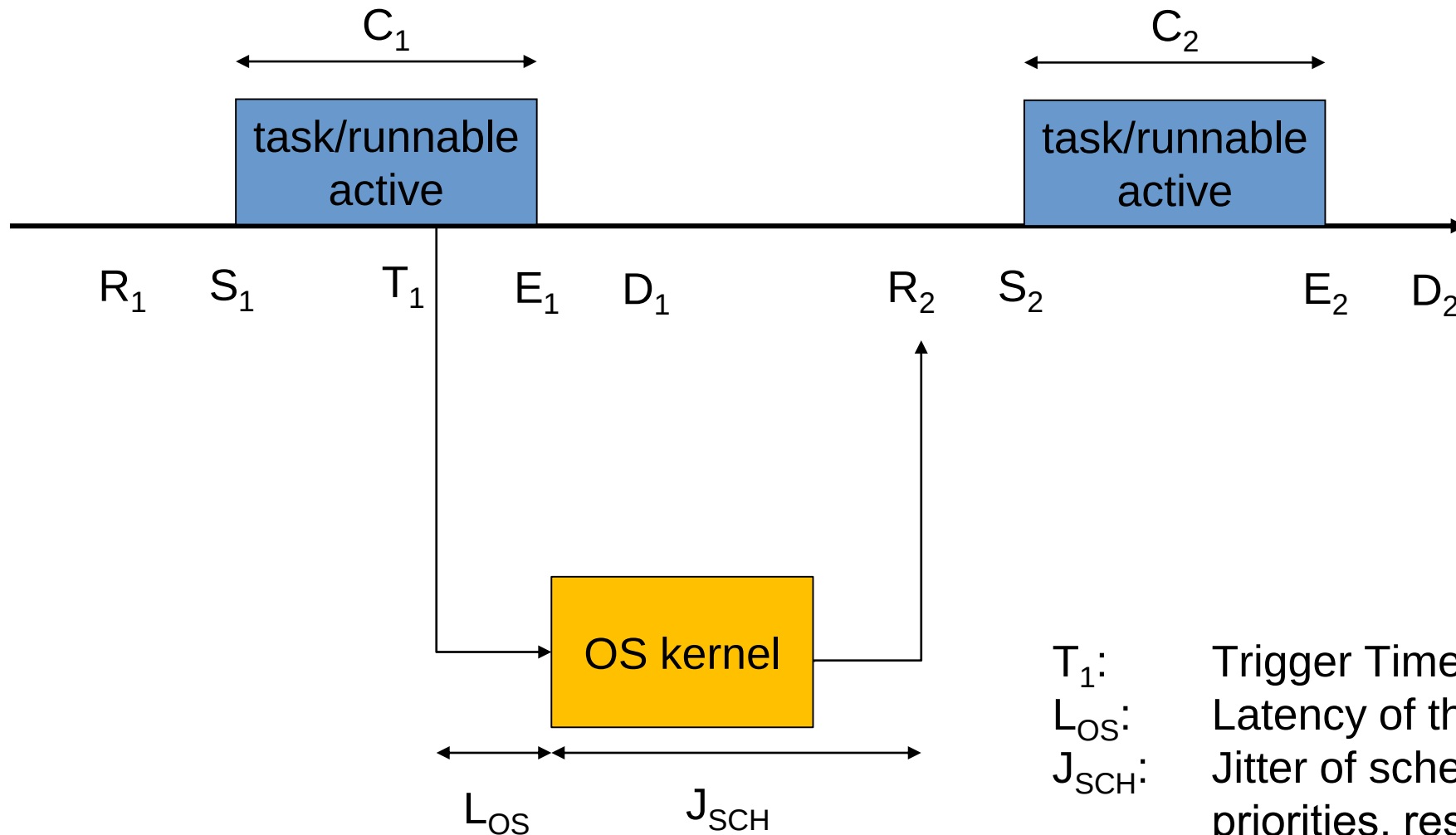
From a cyclic perspective



here there is possibility of having jitter between release time.

- F : (intended) Cycle Time
- J_S : Jitter of the starttime = $S_n - R_n$
- J_C : Jitter of the computation time = $C_n - C_{n-1}$
- J_E : Jitter of the endtime = $D_n - E_n$

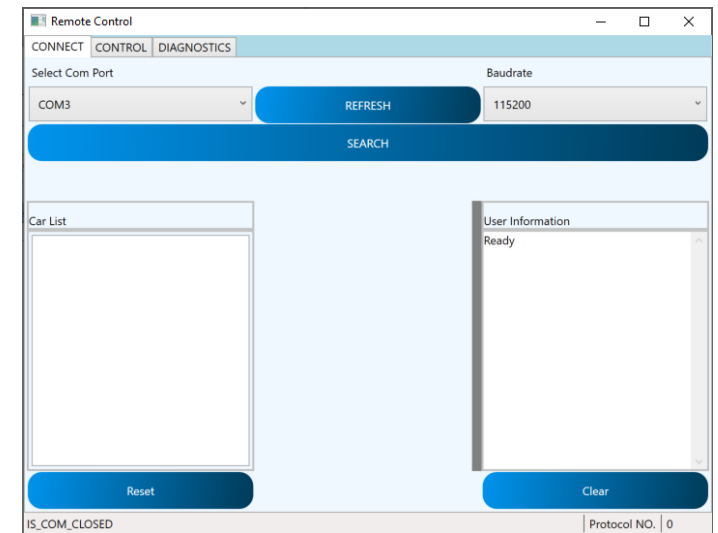
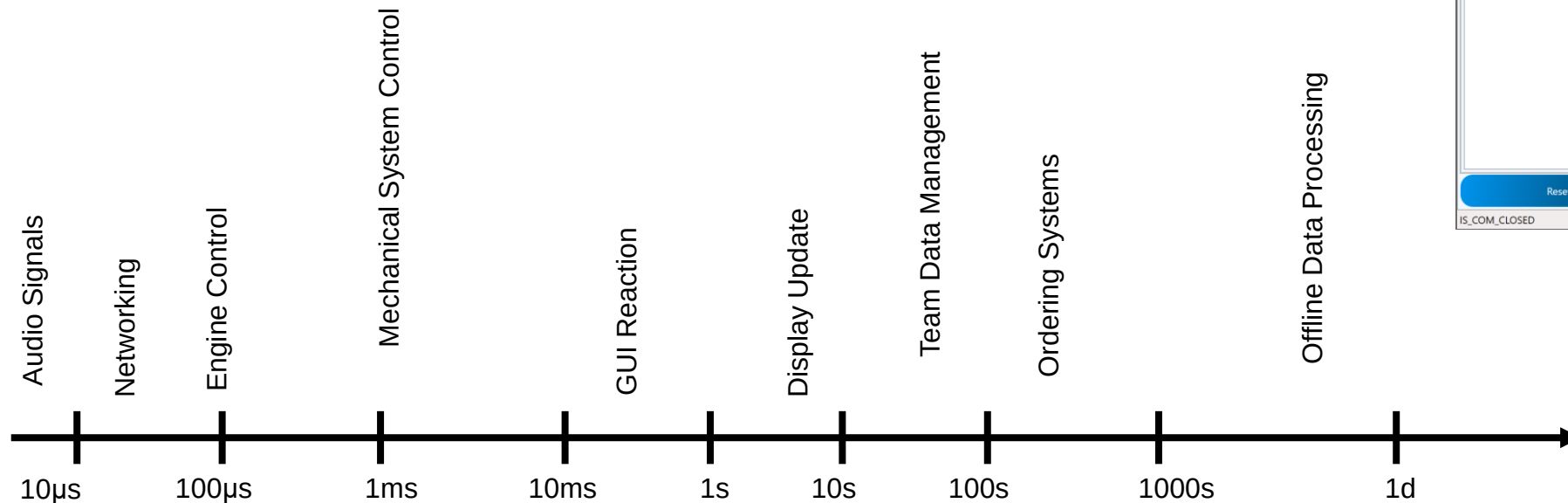
From an event driven perspective



T_1 : Trigger Timer
 L_{OS} : Latency of the OS receiving the event
 J_{SCH} : Jitter of scheduling algorithm (due to priorities, resources, interrupts,...)

Problem: Bad design of start time / cyclic frequency / long latencies

- ✓ Slow system response
- ✓ Wrong user actions (pressing a button several times)
- ✓ Instable control loops



Problem: Deadline not met

Missed deadlines typically result in a cascade of timing problems usually crashing the system!

- You miss the alarm clock (10min late)
- As a result, you miss the train (60min late)
- As a result, you miss the plane to your vacation (1 day late)
- As a result, your hotel room has been given to another guest (CRASH)
- ➔ knowing and considering timing constraints is of fundamental importance when designing a realtime system!

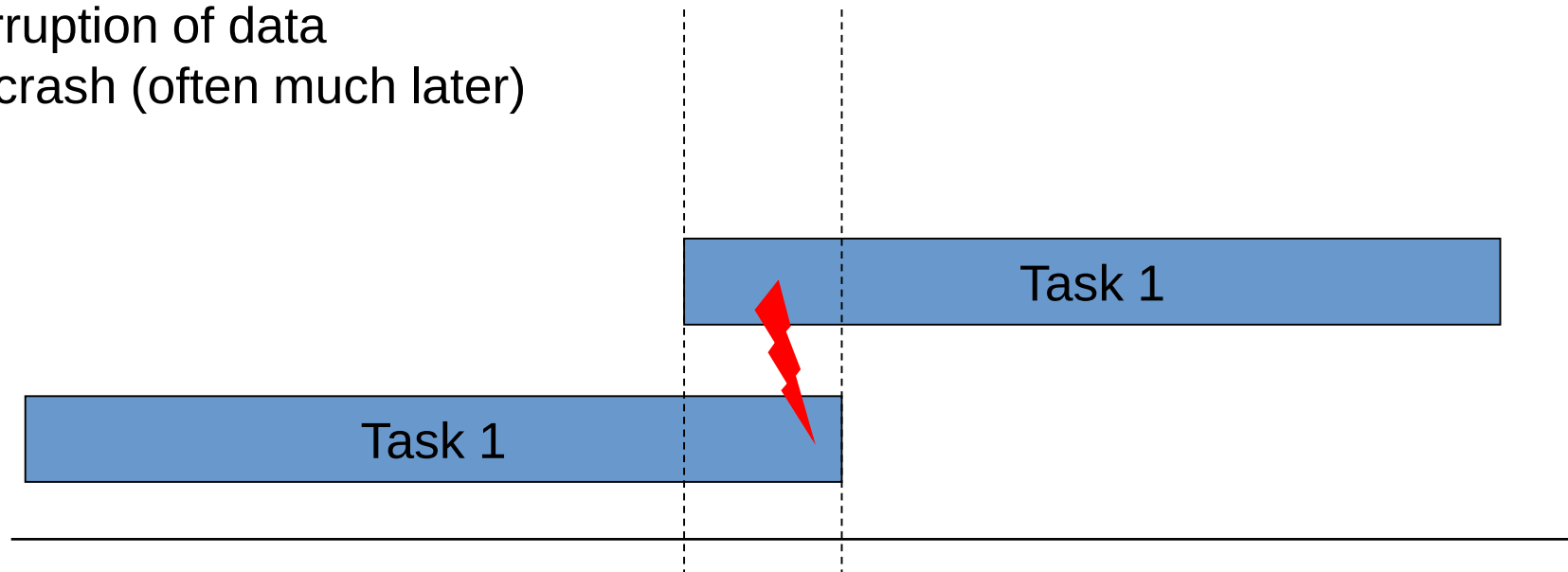
Problem: Deadline not met

Typical effect of deadlines which have not been met: overrun conditions.

- Cascading Interrupt Service Routines
- Tasks are called again before they have been finished
- Calling sequence of runnables corrupted

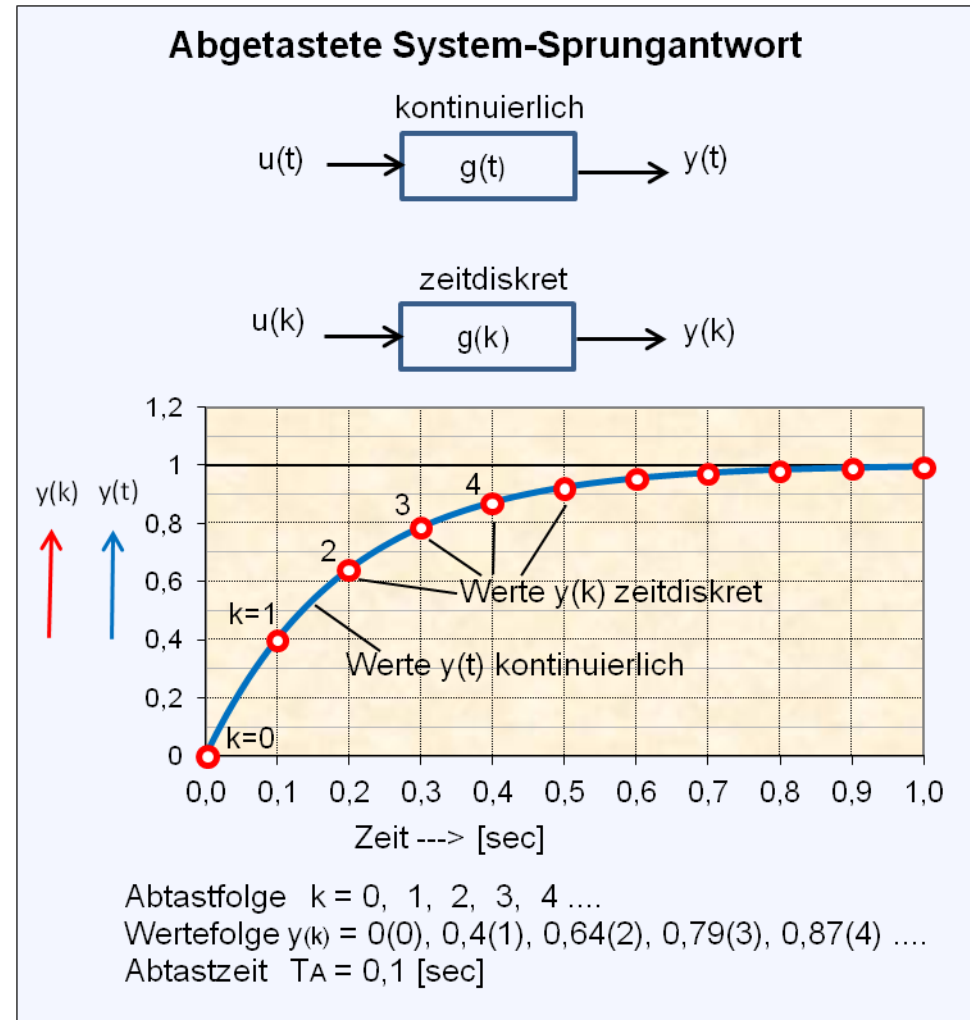
Usually resulting in

- Unexpected execution sequences
- Loss / corruption of data
- Program crash (often much later)



Problem: Jitter

- Instability control loops
- Worst case – dangerous behavior / instability of the actuator



A simple sample...

- We want to implement a closed loop control system to control an engine
- The user enters the data via a potentiometer
- The control algorithm translates the data into a targetvalue
- The engine component translates the target value into a controlvalue
- And reads back the current value (closed loop control)
- Furthermore we want to add a UART port, which allows us to set some more process data (e.g. max value)

What kind of realtime constraints do we have to consider? Where do they have to be considered?



Timing Errors are System Errors

Timing are System Errors which are hard to track/ debug.

A well-defined segmentation of the project effort ensures system modularity. Each task forms a separate, distinct program module. At implementation time each module and its inputs and outputs are well defined, there is no confusion in the intended interface with other system modules. At checkout time the integrity of the module is tested independantly; there are few scheduling problems in synchronizing the completion of several tasks before checkout can begin. Finally, the system is maintained in modular fashion; system errors and deficiencies can be traced to specific system modules, thus limiting the scope of detailed error searching.

Gouthier and Pont, 1970

Forget the weekend, the next week, the next month...

Changes often have a strange and hard to understand impact...

Realtime mystery and tales

Cyclic Way is more deterministic than random event driven approaches

True or false?

- 1) ■ The development of realtime programs explicitly means „performance engineering“
- 2) ■ The ensurance of time constraints is handled by the OS, not by the application programmer false
- 3) ■ Random events should be processed as fast as possible (get the data, store the data, get out) TRUE
- 4) ■ Realtime programming implies multitasking.
- 5) ■ Only the highest priority task will have a deterministic realtime behaviour. TRUE
- 6) ■ If we have implemented a proper scheduling strategy, the realtime behaviour will be independant of the CPU load. TRUE
- 7) ■ The development of realtime systems is the task of gurus, it is far too complicated for normal developers

2) Time constraints are handled by application programmers.

3) Random events are interfering with cyclic behavior of the systems. They are naturally causing jitter and latencies. So they should be as short as possible.

5) If it is true, if there are no interrupts active. Otherwise it is false, lower priority task will never have a deterministic behavior.

6) Usually utilization is below 80%.

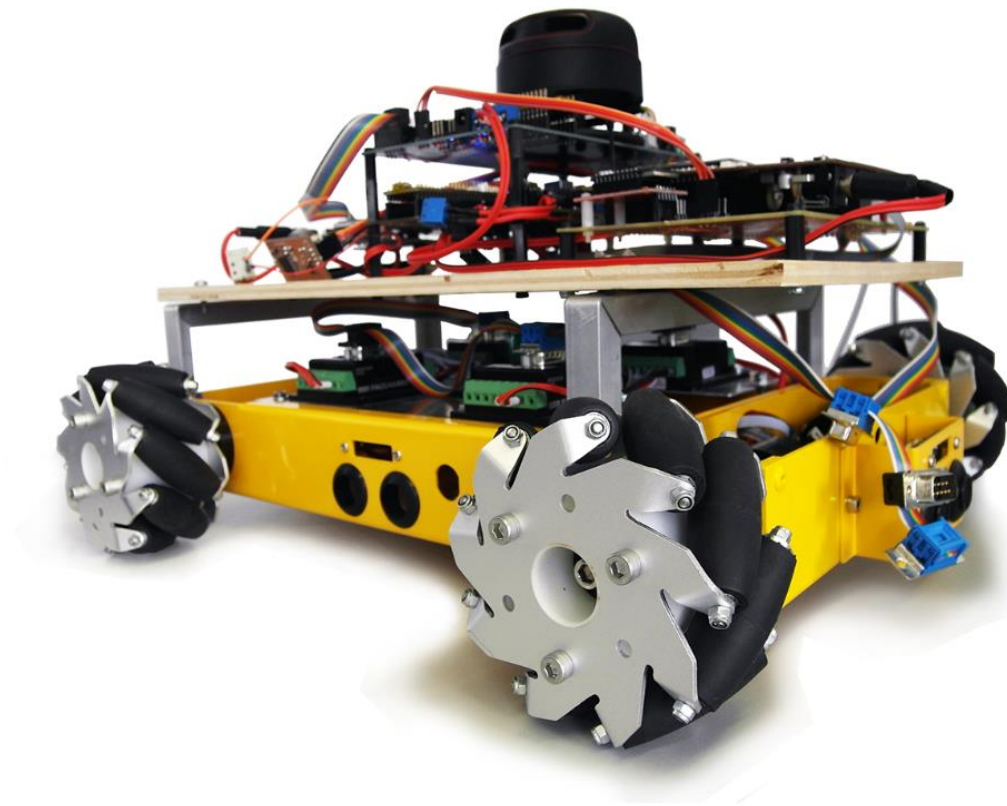
Hard and soft realtime criteria

Typical classification:

- Category A – Hard Realtime
 - Realtime criteria's must be met, otherwise the system fails
- Category B – Realtime *→ This is solvable if ERROR handling is done properly*
 - Realtime violations result in error, but system recovers (e.g. retry after loss of data)
- Category C – Soft realtime
 - Realtime violatation lead to minor impression errors, e.g. reaction time after pressing a button

Impact on priority and timing supervision

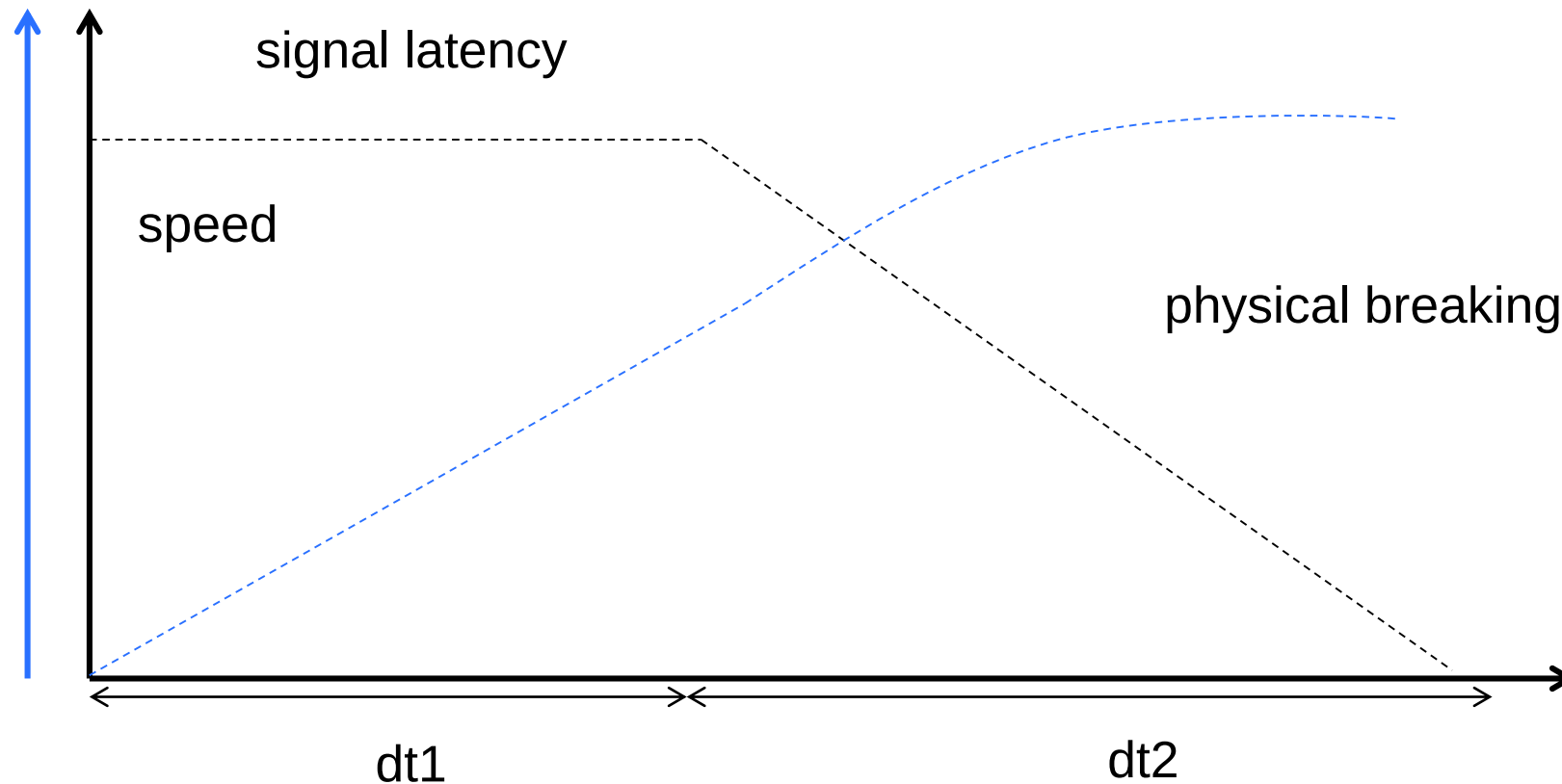
Realtime constraints in an embedded system



Timing Decomposition

- The car drives with 1 m/s
- Upon detection of an obstacle by the ultrasonic sensor, the car has to stop within 0.5m

distance



Timing Decomposition

Signal Latency

Which delays do we have? Let's follow the signal

- Scanning frequency of the Lidar: 5Hz → 200ms
- Processing of the data on the Lidar PSOC: ca. 1ms
- Transmission over UART: 1 Byte @115kB = ca. 0,1ms
- Processing on the AURIX: 100ms runnable
- Transmission to the engine via CAN: ca. 0,5ms

- Max latency: 302ms → 0,302m

Timing Decomposition

Physical breaking

Let's assume a linear breaking behavior

$$v(t) = v_0 - at$$

Stopped means $v(t)=0$

$$t = \frac{v_0}{a} = \frac{1 \frac{m}{s}}{2 \frac{m}{s^2}} = 500ms$$

Some math...

$$d = \int_0^{dt1} v_0 dt + \int_0^{dt2} (v_0 - at) dt \rightarrow d = \frac{v_0^2}{2a} + v_0 dt_1$$

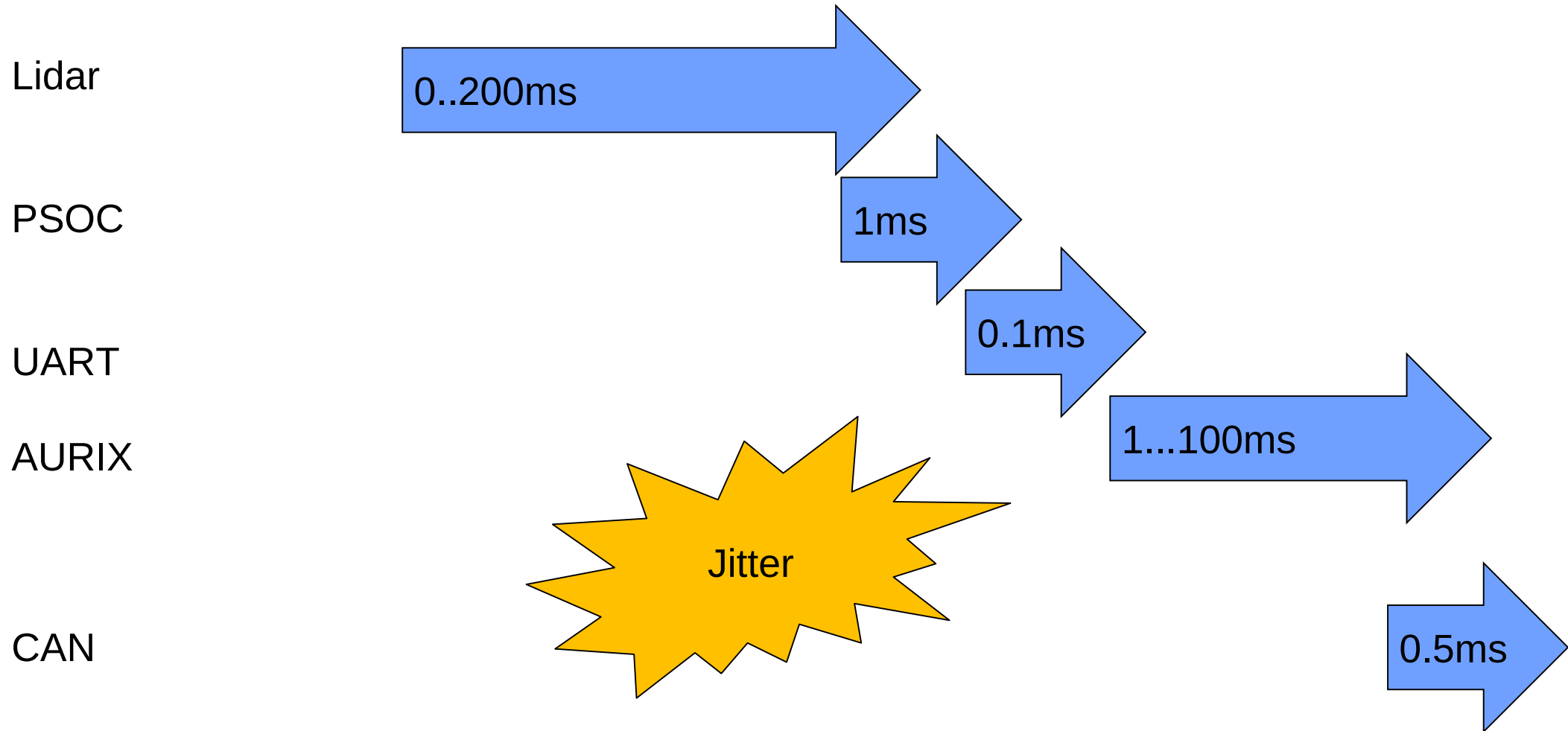


0,302m

0,25m

0,552m

A closer look at the signal latency



Timing Decomposition

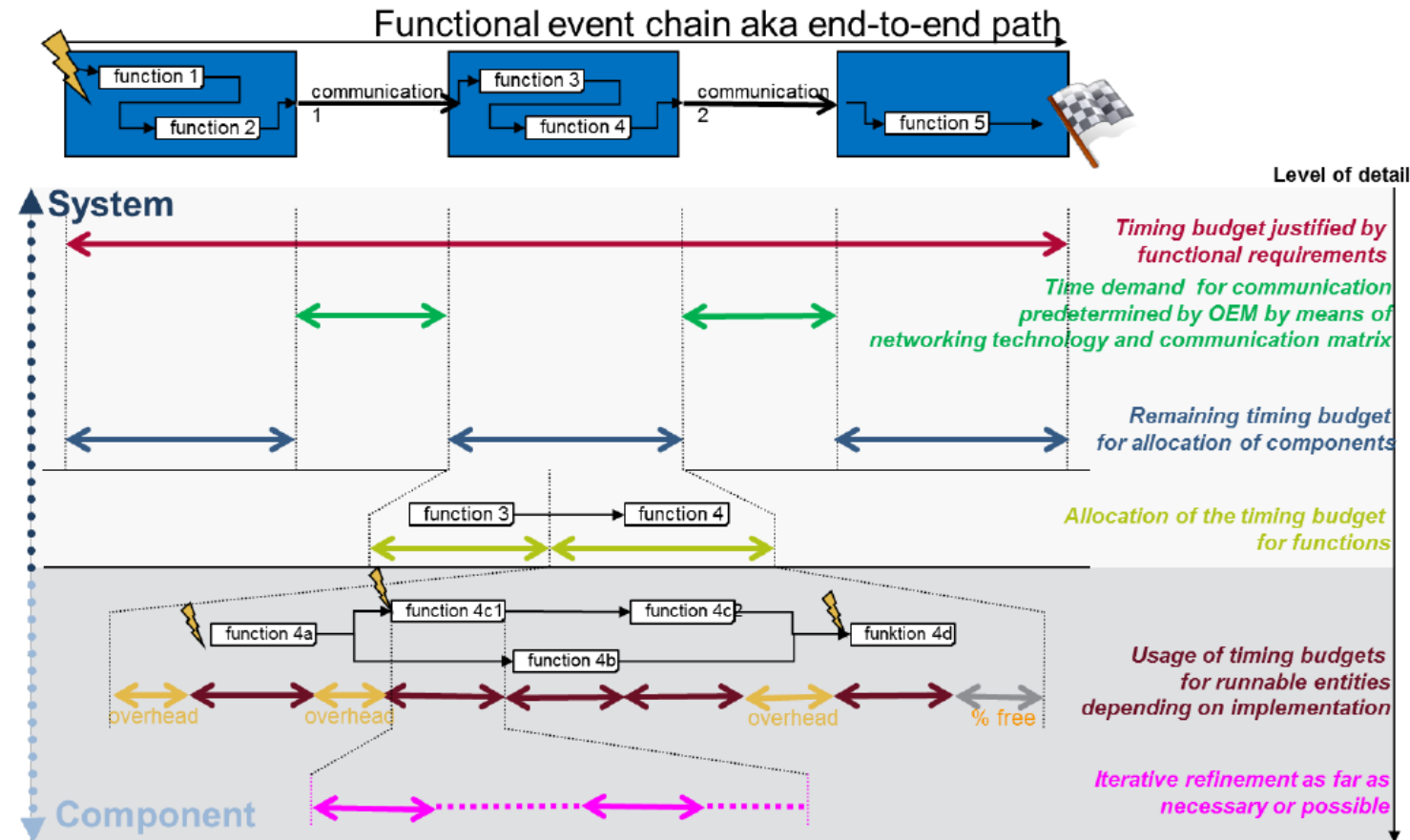
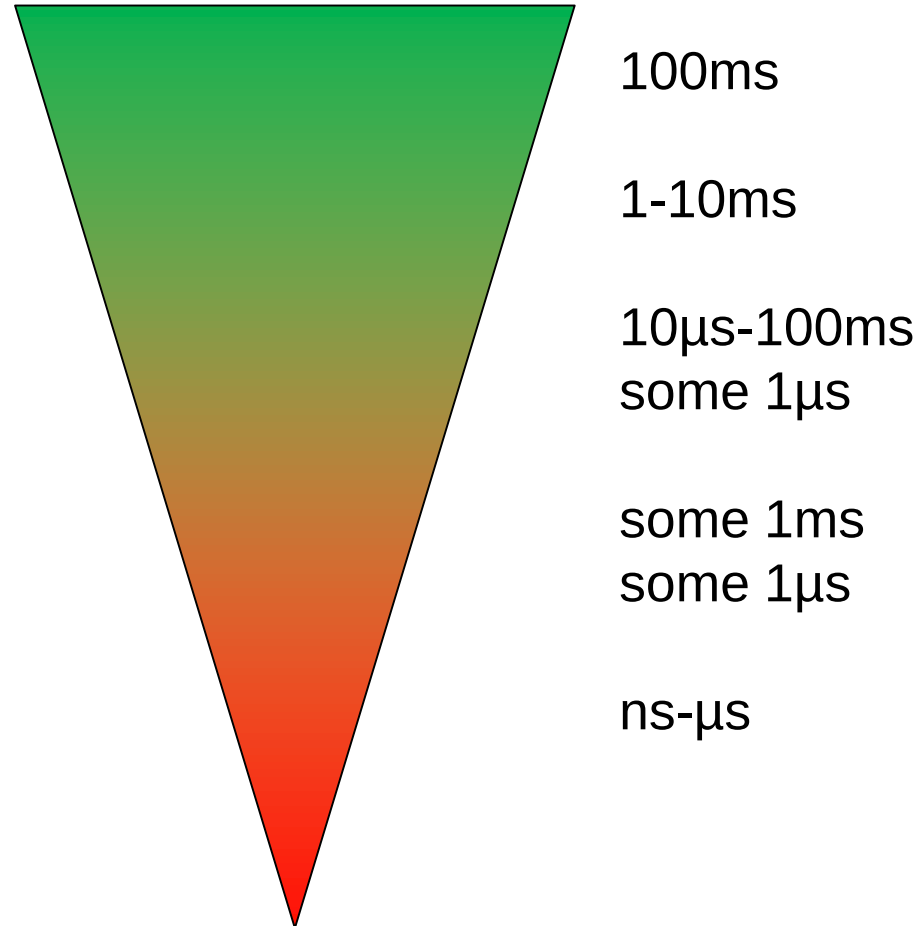


Figure 2.3: Iterative and hierarchical top down budgeting of timing requirements corresponding to response times

Source:Autosar

High level view

- Mechanical delays *highest in 100ms.*
- Communication delays *in ms*
- Software delays *$\mu s - ms$.*
 - Task activation
 - Interrupts
 - Functions processing „large data“ *μs .*
 - Other functions
- Hardware delays *lowest μs*



Designing the timing behaviour

Time Budget

- CPU Load
- WCET

Latencies

- Mechanical
- Communication
- OS Config

Jitter

- Interrupts
- OS
 - Asynchronous events
 - Semaphores
- Communication
- State depending WCET

Time Budget: Feasibility and Stability

- A task is a function controlled by the scheduler
- The total of all controlled tasks is called the **taskset**
- The scheduler controls the resource CPU-time
- The scheduling algorithm defines, which task from the taskset receives at a certain time the CPU resources for a certain time
- A taskset, which can be executed under all given circumstances is called **feasible**
- A scheduling algorithm, which under overload conditions ensures that at least a critical subset of tasks is executed is called **stable**
- A scheduler consists of a **planning algorithm** and a **dispatching algorithm**

Time Budget: A first rough estimation

Basic idea of a static, mathematical approach:

- Considering n Tasks which have a run time of C_n and are called in T_n intervals (or $1/T_n$ frequency)
- The taskset is stable, as long as $U(n) = \sum_n \frac{C_n}{T_n} \leq 1$
- Example:
 - Task 1: $C_1 = 30\text{ms}$, $T_1 = 60\text{ms}$
 - Task 2: $C_2 = 10\text{ms}$, $T_2 = 60\text{ms}$
 - $\rightarrow U(2) = 1/2 + 1/6 = 2/3 < 1 \rightarrow \text{stable}$
- Simply for cyclic tasks, more complex for event driven tasks (average event frequency needs to be known)

Cycle times are known, the interesting parameter is the computation time.

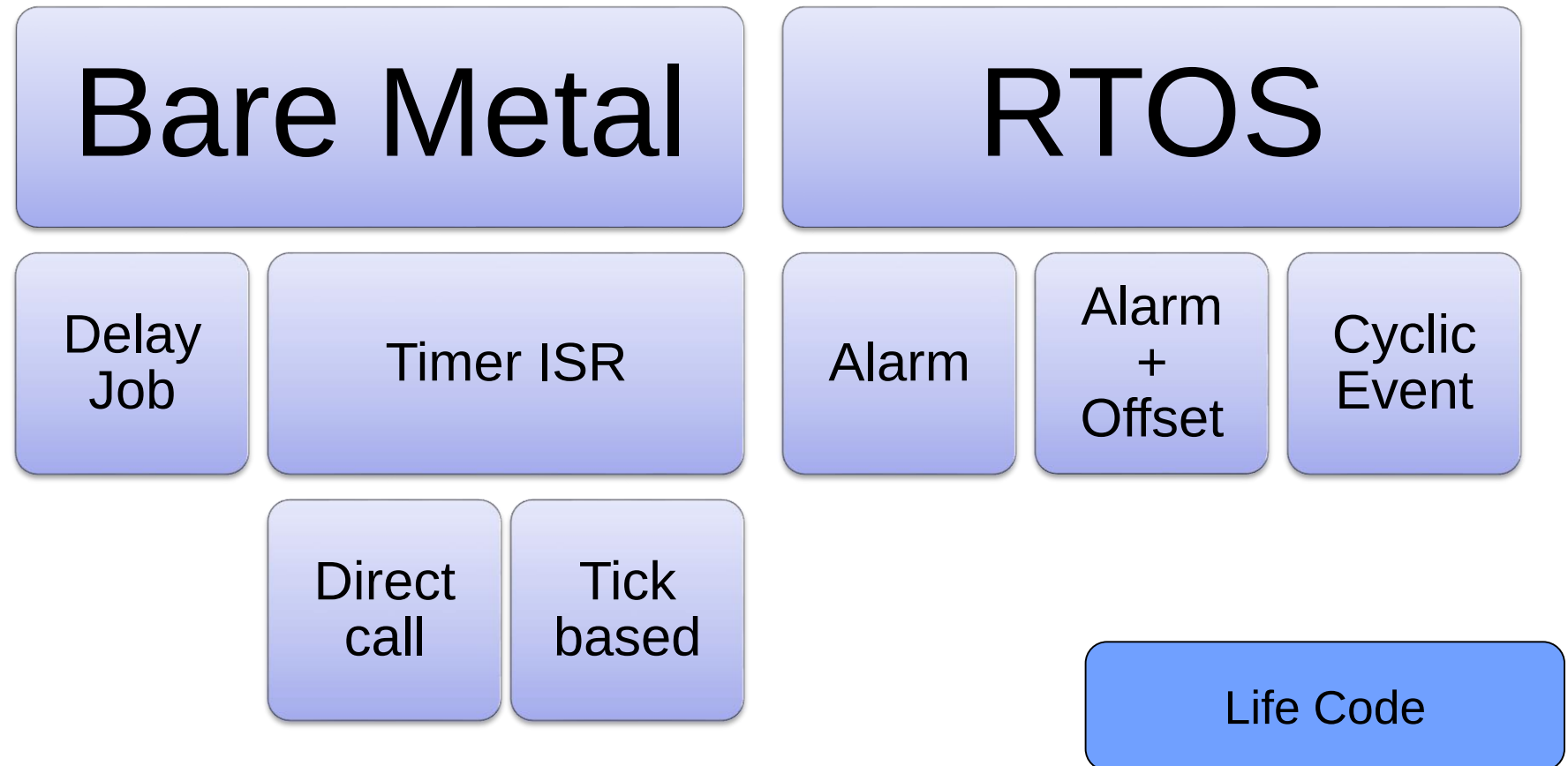
Latencies and Jitter

- Mechanical latencies can be measured
- Communication latencies can be (roughly) calculated considering bandwidth and protocol size.
- OS priorities are (mostly) static and don't have a dynamic impact as long the system is stable or at least feasible. They may however affect the execution order!
- Jitter is very hard to measure, as many parameters have an influence
- Typical approach: stresstest scenario and measure the reallife execution time between relevant sequence points.

Enough Theory

Let's realize a cyclic job...and compare the different implementation patterns concerning

- Precision
- Jitter
- Latencies
- Races



Delay Job

Concept

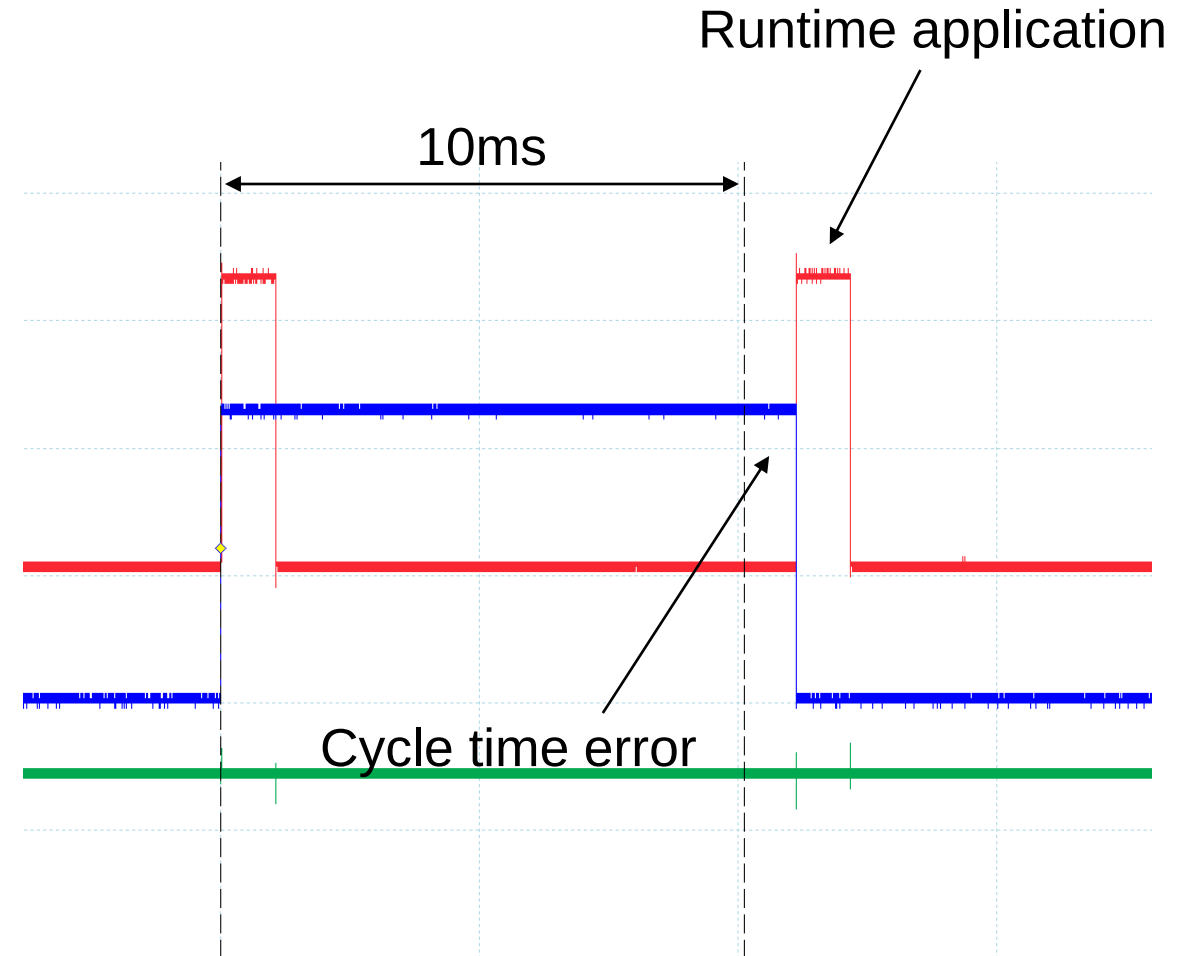
- Implement a superloop with a blocking delay job

Advantages

- Very simple

Disadvantages

- Cycle time is influenced by application runtime
- Most of the CPU time is used up busy waiting



Timer ISR – Direct Call

Concept

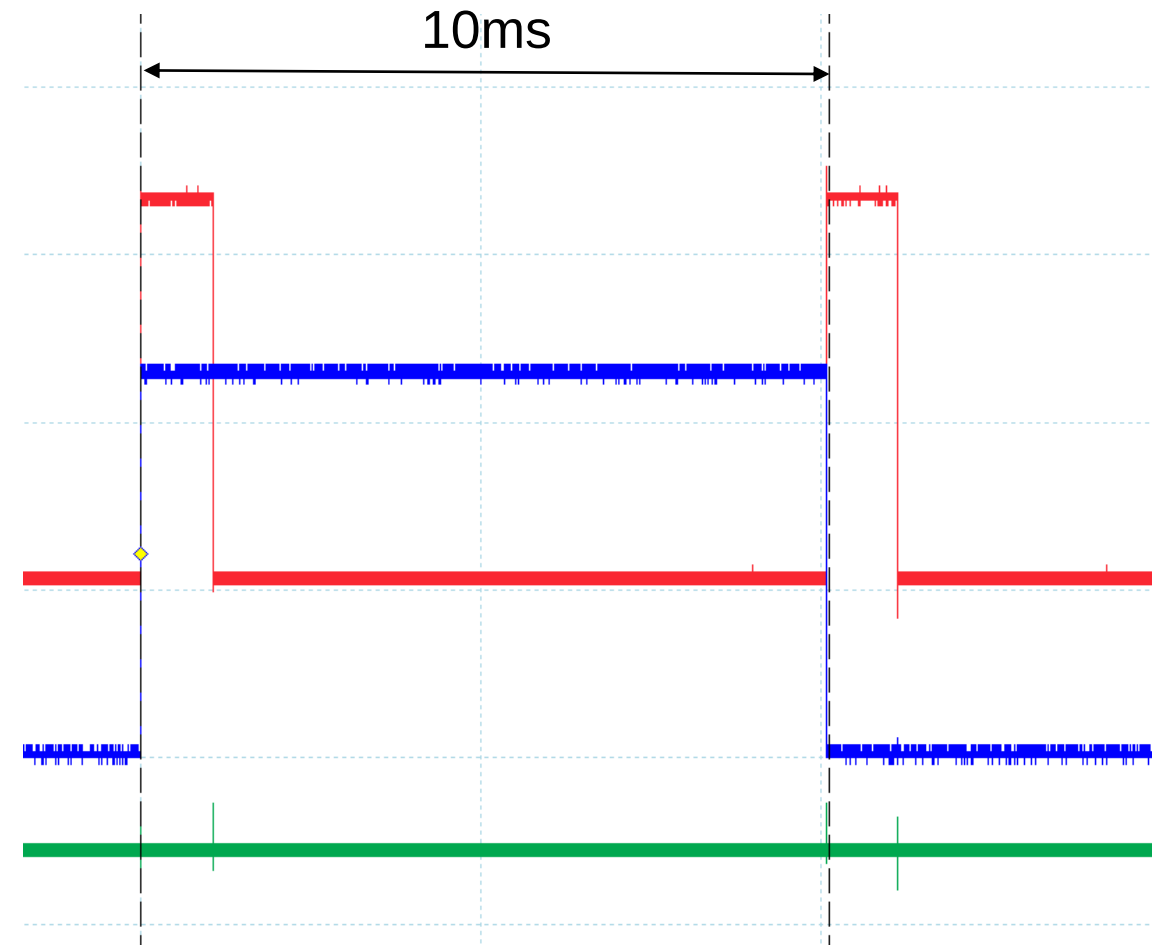
- Cyclic code is executed in a timer ISR

Advantage

- Still pretty simple
- Precise cycle time independent from application runtime
- No busy waiting, superloop can be used for background job

Disadvantage

- Code is executed in ISR context
- Other interrupts might be blocked
- Nested interrupts may occur if runtime is too long



Timer ISR - Tick

Concept

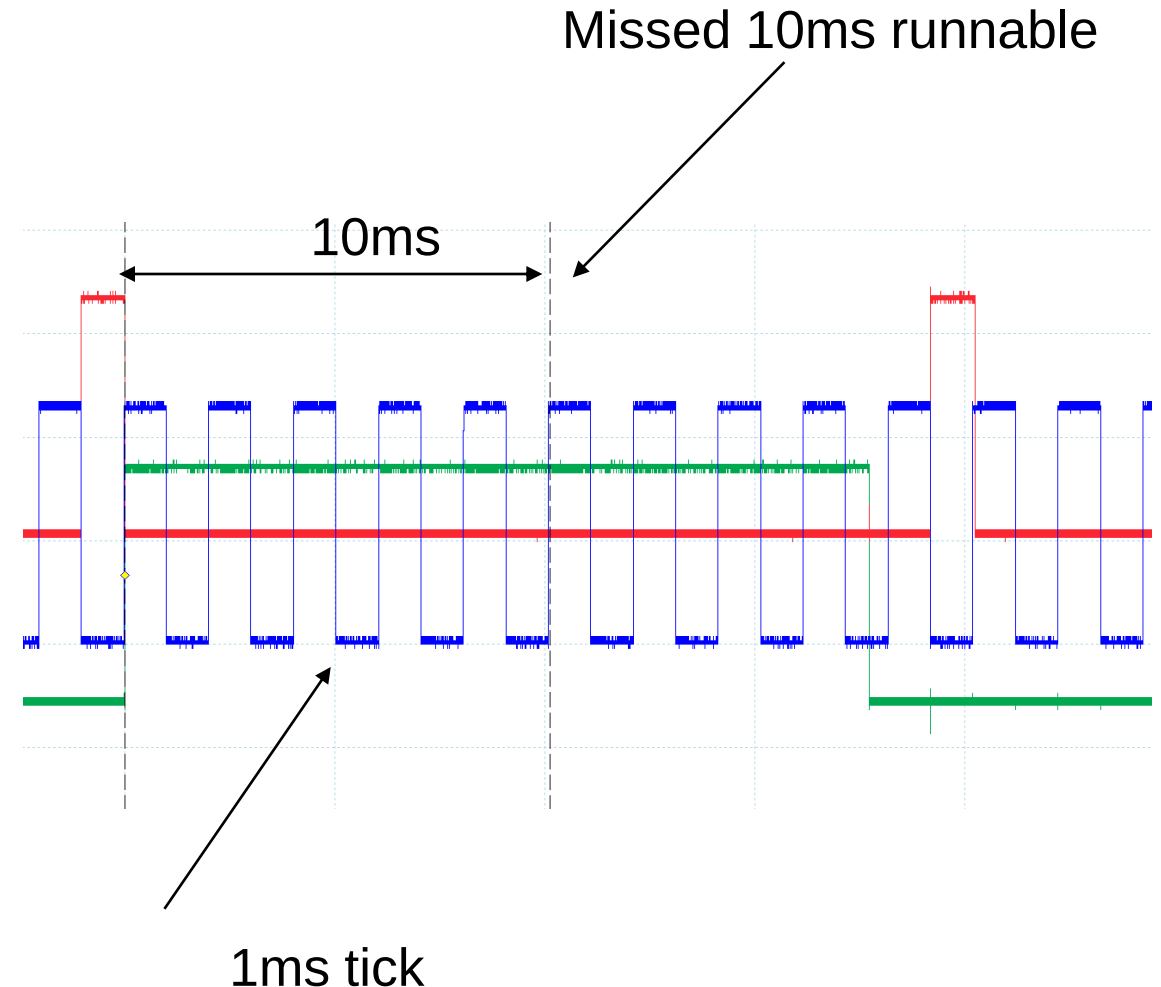
- Timer ISR creates tick which is processed in superloop

Advantage

- Code is executed in application context
- Precise timing
- Different cycle times can be executed simultaneously
- No busy waiting, superloop can be used for background job

Disadvantage

- Bit more complex
- Runnables having a longer cycle time (and runtime) may block faster runables



OS Alarm

Concept

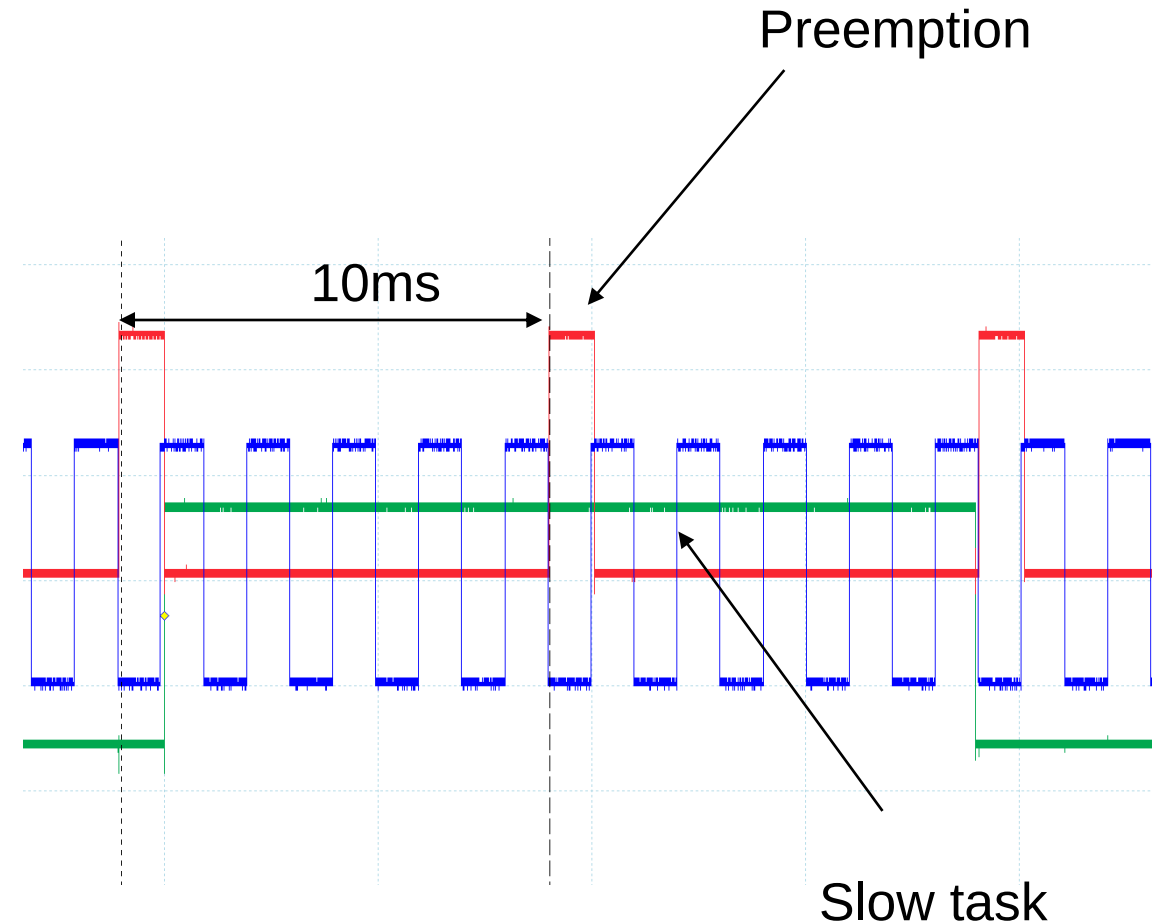
- Similar to the Tick approach presented before, but the OS supports different (preemptive) priorities

Advantage

- Fast task preempts slow task

Disadvantage

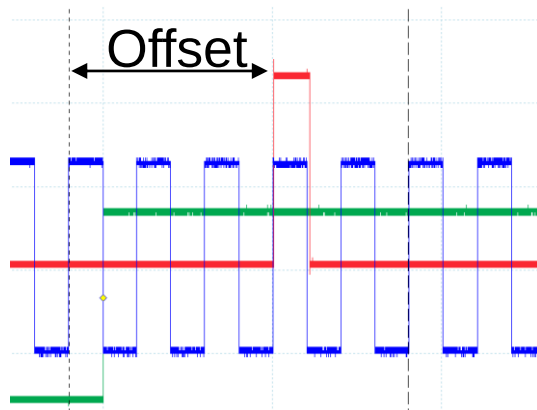
- Higher complexity



OS Alarm Finetuning

Alarm offset parameter

- By default, the highest priority task will start first if several tasks become active at the same timeslot
- In order to ensure a more constant CPU load, it is recommended to use the first offset parameter of the SetRelAlarm API



Event versus task activation

- Creating an individual task for every cycle time typically is too expensive.
- Tasks you be used if you require priority based preemption
- Use a software tick instead.
- If you need to mix cyclic and asynchronous activations, an alarm firing an event might be a good choice.

Application Execution Time

No matter which approach we selected: A too long execution time of a runnable creates problems!



Worst Case Execution Time

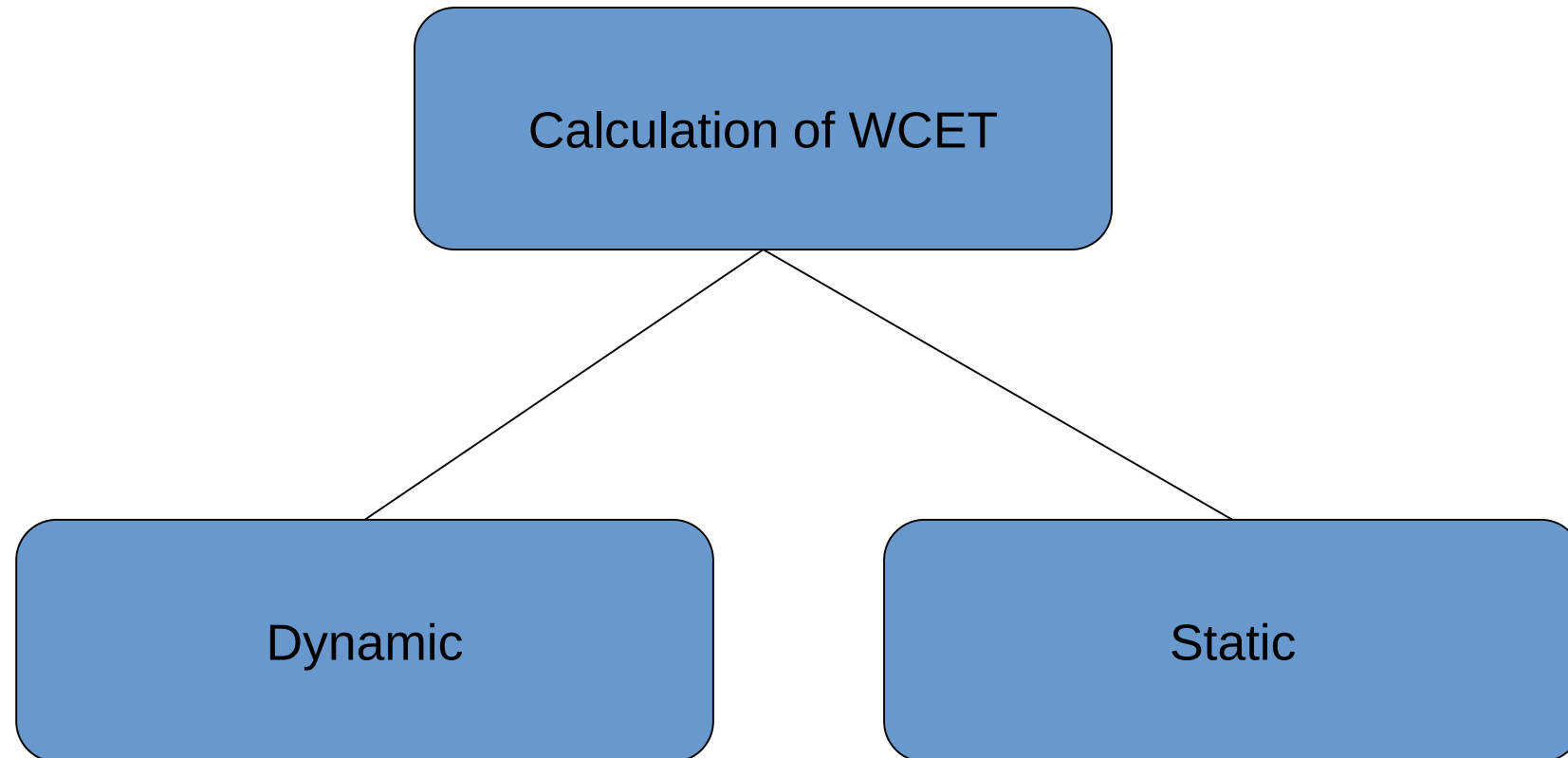
Definition

- The **worst-case execution time (WCET)** of a computational task is the maximum length of time the task could take to execute on a specific hardware platform.
- The **best-case execution time (BCET)** of a task is the minimum length the task will run on a dedicated hardware platform.
- **Low level analysis** considers the execution time of a single task without interruption or blocking
- **System level analysis** considers the interaction between tasks and interrupts.

Impacting factors

- Performance and Timing Measurement of integrated SW Systems must be performed in order to verify, that the performance requirements are met.
- Performance and Timing Measurement must be done under typical and under maximal load.
- Several aspects must be considered:
 - run times (minimal, average, maximal) of: ISR, tasks, containers, functions
 - latency (minimal, average, maximal) of: task activation, ISR processing (pending ISR), CAN message processing, OS Event processing, Event handler processing
 - locks (how long, how often): ISR, OS resources, module specific semaphores
 - resource usage (% of total): stacks, CPU, network
 - schedulability: are there any deadlines missed?

Worst Case Execution Time

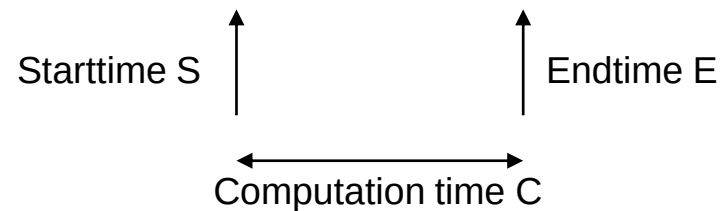
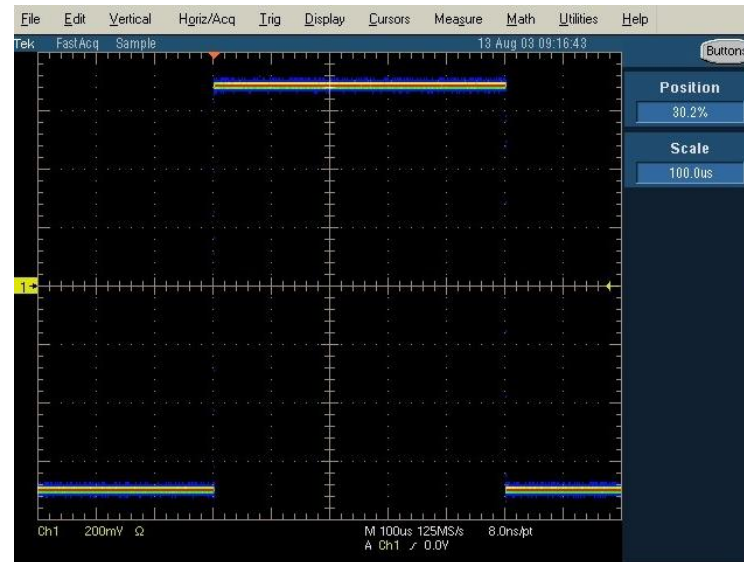
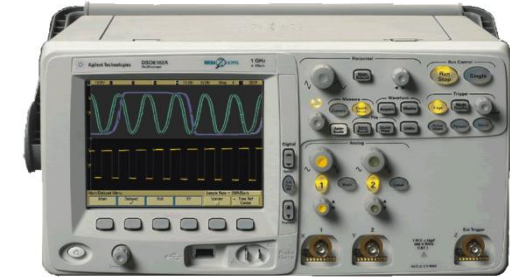


Dynamic Measurement of WCET

- Dynamic measurement is typically performed as „stresstest“.
 - Testcases are not selected to cover a certain functionality (functional testing) but are selected on the basis of the longest runtime.
 - Critical parameters, high peripheral loads (e.g. CAN)
- The quality of the measurement depends heavily on the experience of the tester.
- Advantages:
 - Good testcases generate a realistic result.
 - Always possible
- Disadvantages:
 - Uncertainty: did we really measure the worst case?
 - High (manual) test effort

Tools for Dynamic Measurement Scope

- Scope connected to a binary port
- Software sets and resets the port



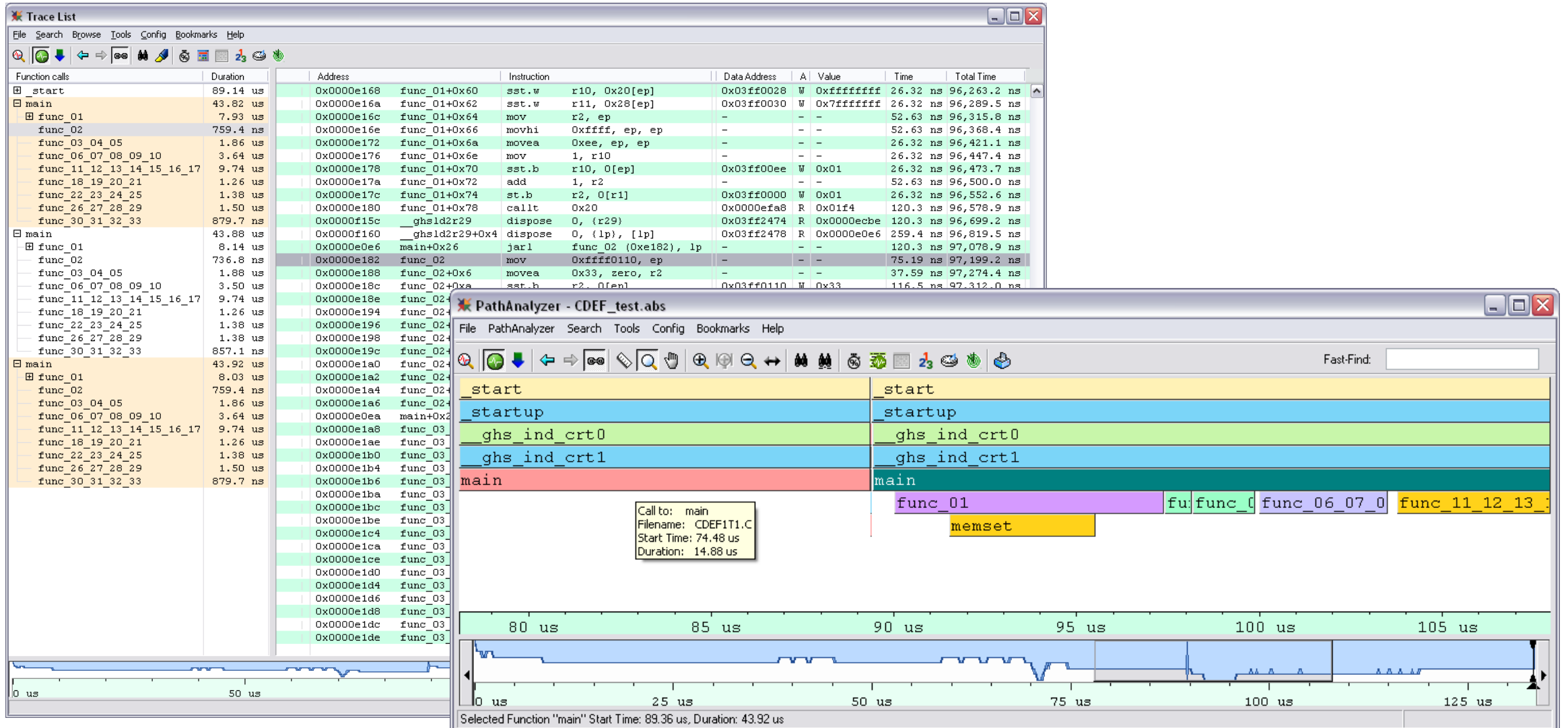
Tools for Dynamic Measurement Emulator

- Timing is measured by the emulator
- Using additional profiling memory
- Or trace buffer memory



Greenhills Supertrace Probe

Tools for Dynamic Measurement Emulator - Profiling



Tools for Dynamic Measurement Tracing

- Timing is measured by additional measurement module
- Measurement data is stored in profiling memory

The screenshot displays the MULTI Debugger interface for the file 'hex\NEC_V850_proj.abs'. The main window shows assembly code with a breakpoint set at line 376. The 'Registers' window at the top right shows the state of the 'ITR_stCommonMeasureCtrl' and 'ITR_stTaskTimeMeasure' structures. The 'Memory' window at the bottom right shows the contents of the 'ITR_stTaskTimeMeasure' structure, which includes various timing and measurement data.

Registers:

- ITR_stCommonMeasureCtrl
 - boStartMeasure: '\0'
 - boMeasureRunning: '^A'
 - boStopMeasure: '\0'
 - boMeasureValid: '\0'

Memory (ITR_stTaskTimeMeasure):

Field	Value
dwRunTimeSumBrutto	4394451
dwRunTimeSumNetto	3867255
dwNumberOfMeasures	120
dwSumRunTime	27459616
dwSumPeriod	32899074
dwMaxPeriod	393344
dwMinPeriod	131198
dwCurrentPeriod	196561
dwCurrentDelaySum	0
dwRunTimeMaxBrutto	65542
dwRunTimeMinBrutto	170
dwRunTimeBrutto	65453
dwRunTimeNetto	65412
dwRunTimeMaxNetto	65500
dwRunTimeMinNetto	129
dwSumActiveCountInst	0
dwSumActiveCount_is	0
dwSumTimeActiveCount	0
wStartTimeBrutto	65389
wStartTimeNetto	65472
wStartTimeRunTime	65425
wStartTimePeriod	65425
wCurrentDelay	0
wStartIntTime	0
wIntTime	0
wMaxActiveCountInst	0
wTimeActiveCount_is	0
wMaxActiveCount_is	0
wCurActiveCount_is	0

Assembly Code:

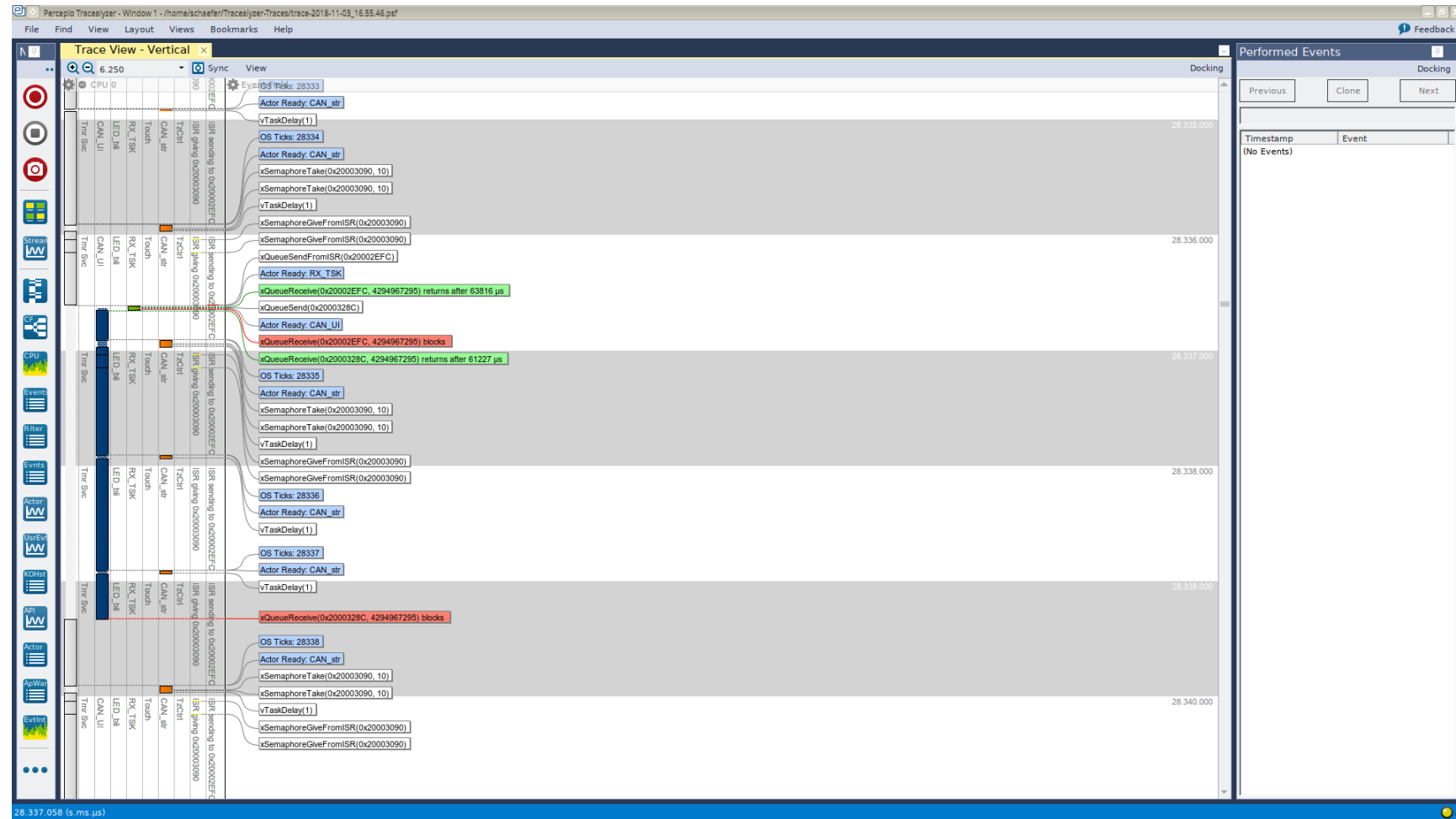
```
370      ..lxtdaG.__start:=0
371      #endif
372
373  1      _start:
374  1      _RESET:
375  2      nop
376  3      nop
377  4      nop
378  5      jr _startup
379  6
380  1      _startup:
381  2
382  3      -- Set an IO Register Base Address
```

Command Window:

```
Source: File: start1c1.850 Proc: RESET
HWBP at reset vector configured.
----- Target initialization done -----
Finished executing setup script.
Downloading program text and data. Please Wait...
Download complete.
running 'hex\NEC_V850_proj.abs'
MULTI>
```

Status: STOPPED

Example Percepio Tracealyzer



Comparison of Dynamic Measurement Methods

Method	Hardware Requirement	Software Requirement	Limitation
Oscilloscope	Scope Binary Pin	Set / Reset Port	Single objects Some Milliseconds
Emulator	Emulator Profiling Memory	None	Single objects Some Milliseconds to seconds
Tracing	Trace Buffer Free running timer	Complex Measurement Module	Complete System Seconds to Hours

Dynamic Measurement

Problem:

- High effort
- Open question: did we really measure the right thing.

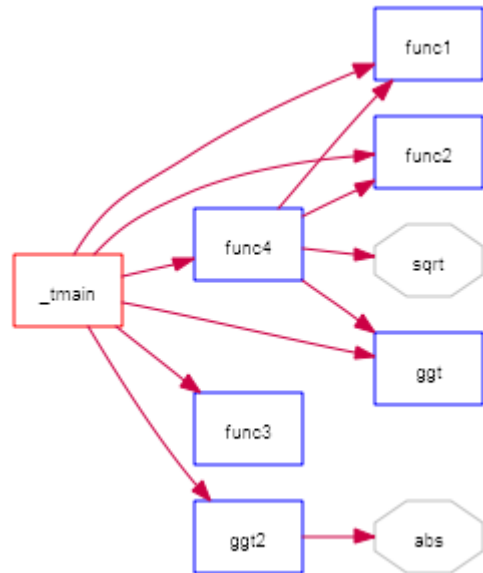
Idea (Dream)

- Press a button and get a reliable answer on the feasibility and stability of the system by a tool
- ➔ Static Analysis

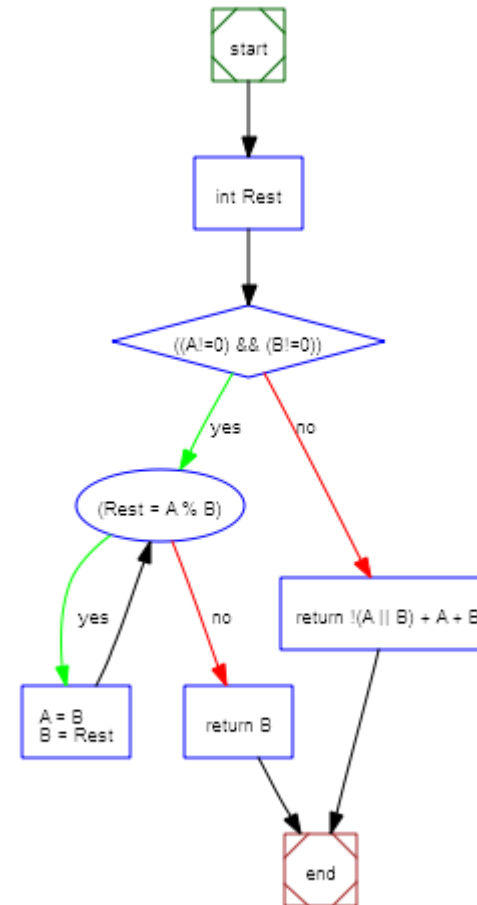
Static Measurement of WCET

- Idea: Use a tool to calculate the cycles of the longest program flow
 - Consideration of loops, descisions etc.
 - Consideration of data values, parameters etc.
 - Consideration of hardware aspects like pipelines, cache etc.
- Advantages:
 - Tool can be integrated into build process (automation)
 - Worst Case Time is calculated
- Disadvantages:
 - High configuration efforts especially for system level analysis
 - Latencies, blocking not considered (e.g. HW failure).
 - Uncertainty: Does the calculated pessimistic time really occur?
 - Pessimistic approach ➔ increasing hardware costs

Static Measurement Code Analysis Tools

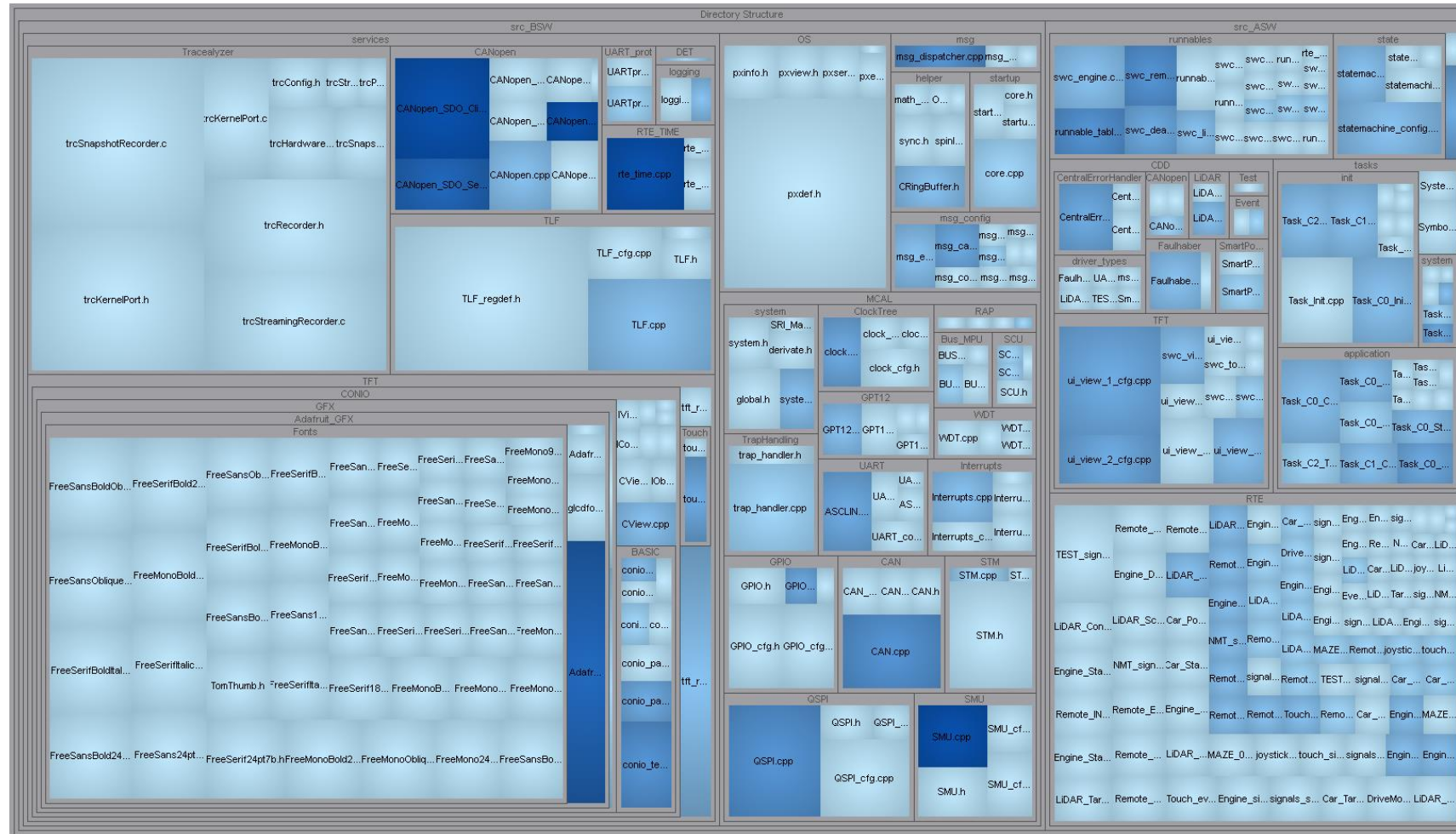


Example Calltree



Example Activity Diagram

Static Measurement Code Analysis Tools - Metrics



More Advanced Tools for Static Measurement

- Simply **measuring** the execution time of a task for a given input is typically **not safe**. It is practically impossible to prove that the conditions leading to maximum execution time are taken into account
- **WCET** of any task/ISR must be known

Tool: aiT
by AbsInt

Tool: SymTA/S
by Syntavision

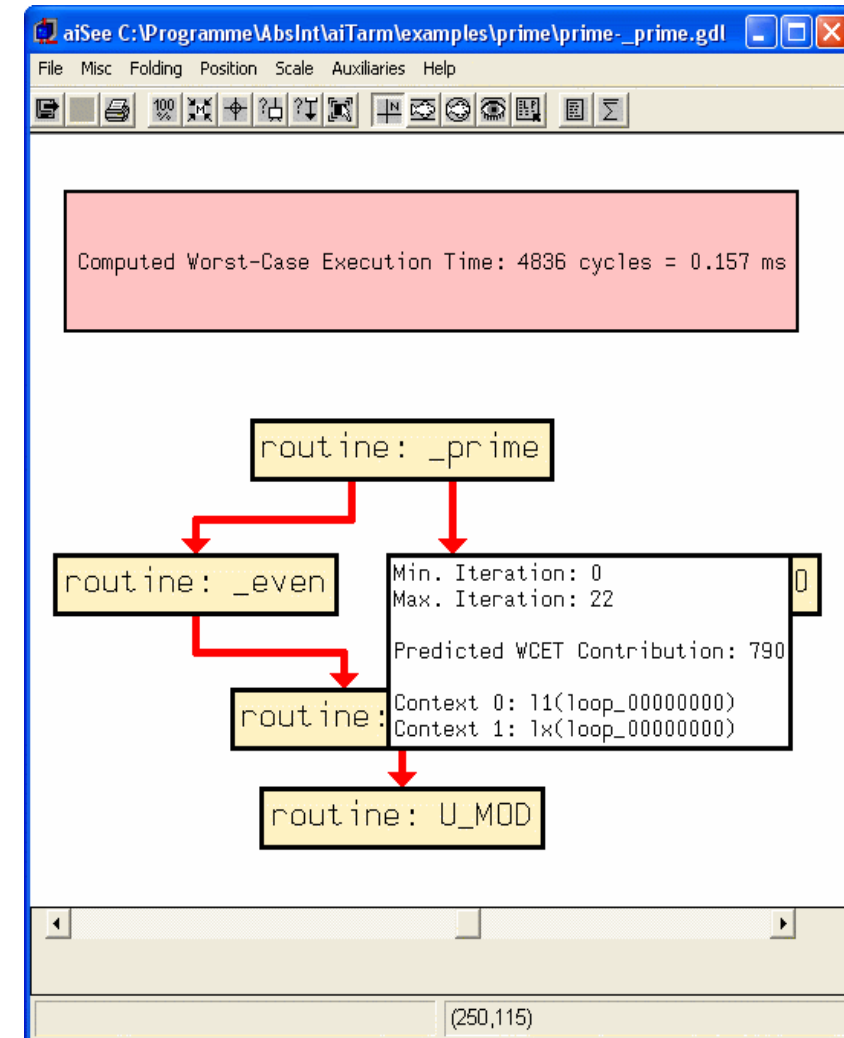
AbsInt aiT: Features and Limitations

- WCET Analysis (without interrupts and task preemption)
- Processor specific tool versions. Currently supported
 - MPC565, MPC755, C16x, ST10, ARM7
 - Planned NEC V850 (End 2008)
- Analysis with aiT is based on:
 - compiled and linked executable
 - start address of the task/isr root function
 - a list of memory areas with minimum and maximum access times
- References: EADS Airbus A380

AbsInt aiT: Features and Limitations

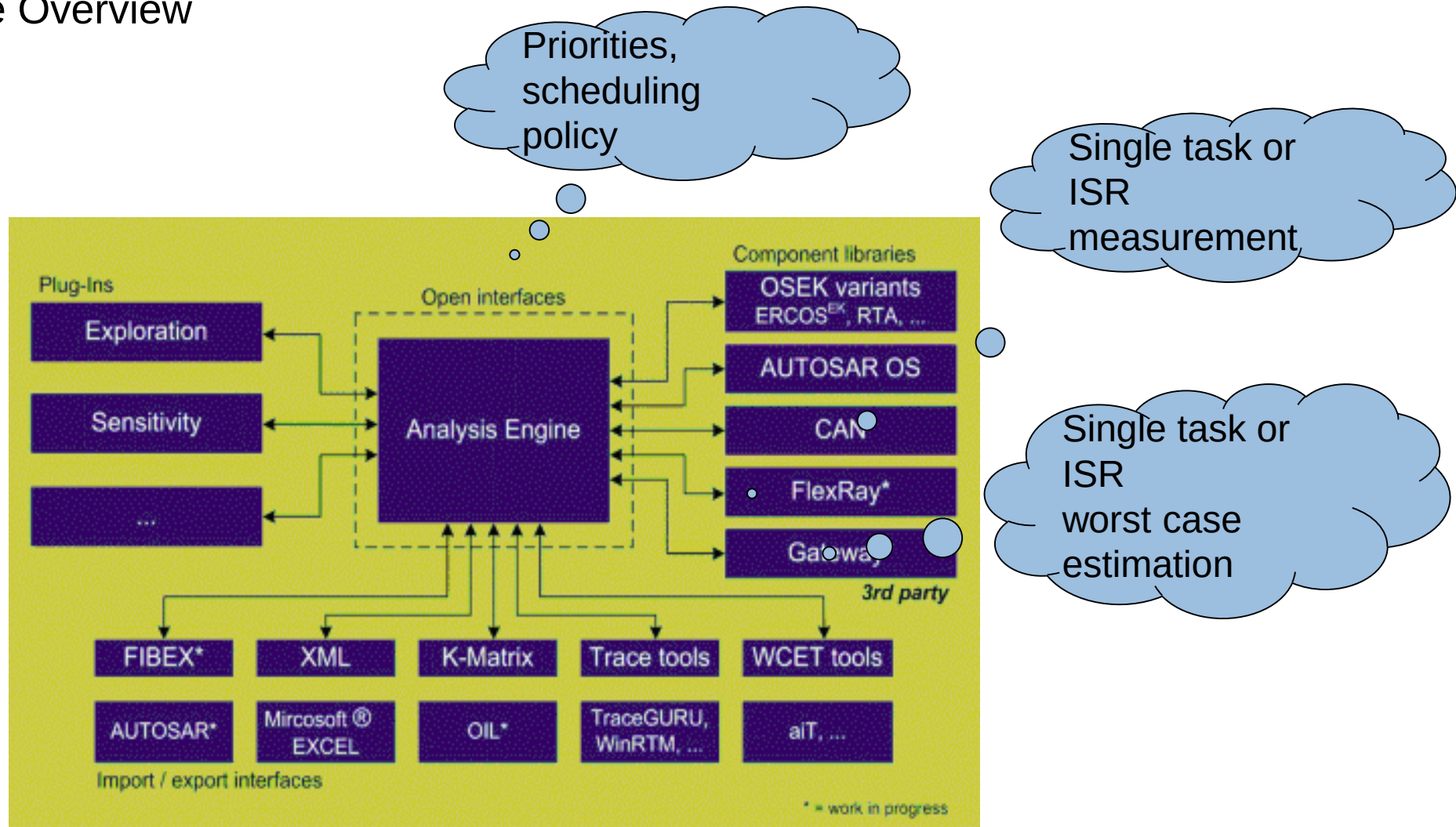
A sample of a simple Worst-Case Execution Time gives:

- the call graph of functions starting from the specified root function
- per function the predicted WCET contribution
- the total number of cycles and the elapsed time.



SymtaVision's Scheduling Analysis Tool

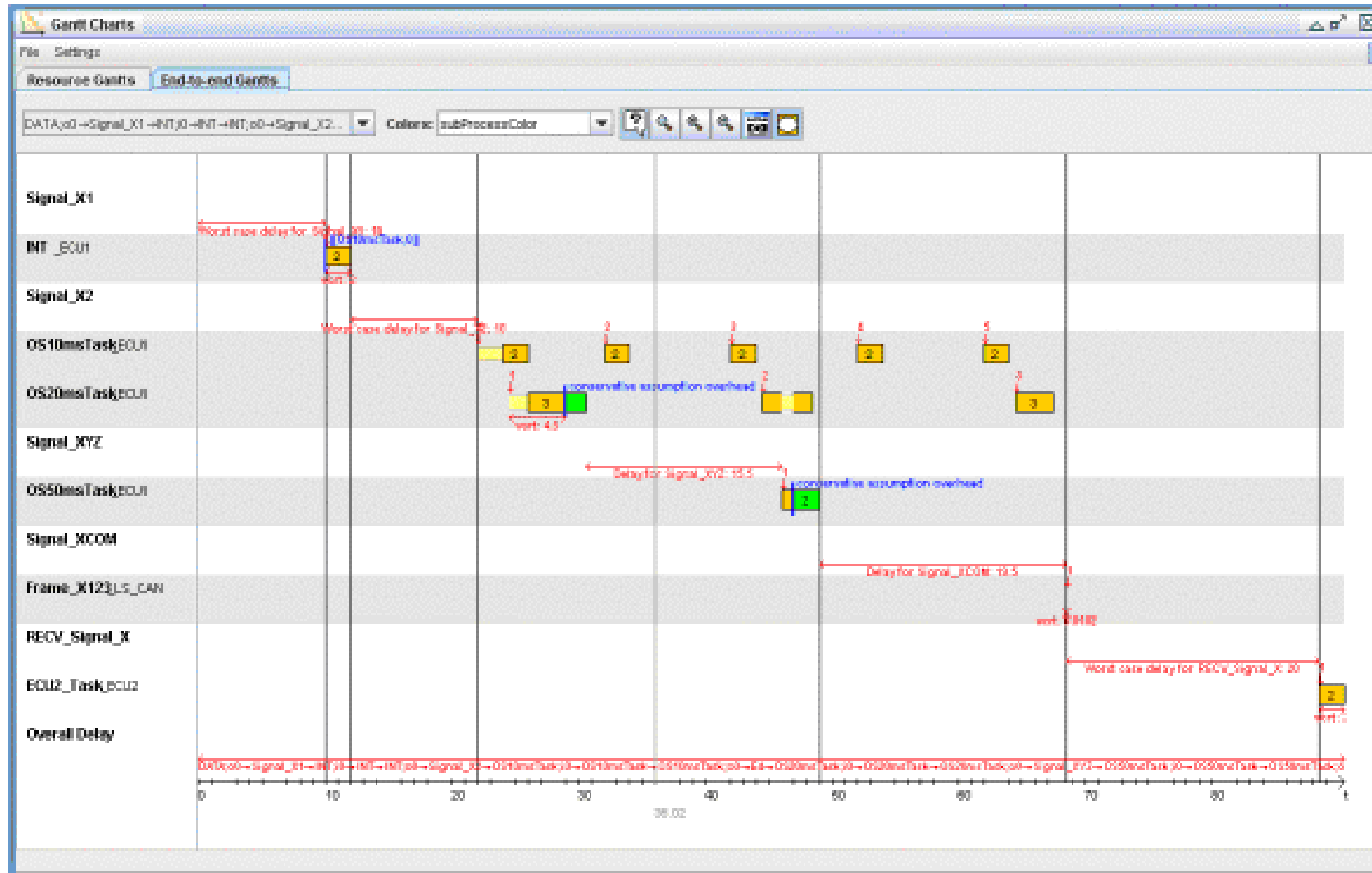
Tool Suite Overview



SymtaVision's Scheduling Analysis Tool

- SymTA/S performs scheduling analysis for
 - CPUs with RTOSes,
 - buses with arbitrating protocols,
 - and systems consisting of multiple resources (CPUs and buses).
- SymTA/S calculates
 - resource loads,
 - worst-case response times for tasks scheduled on CPUs,
 - worst-case transmission times for messages sent via shared buses,
 - end-to-end latencies
- and compares these values against user-specified constraints, e.g. deadlines.
- References: BMW

SymtaVision's Scheduling Analysis Tool



Static versus dynamic measurement

	Measurement	AbsInt aiT	SymTA/Vision
Individual Tasks No interrupts/blocking			
Individual Tasks Incl. interrupts/blocking			
Individual ISR No interrupts/blocking			
Individual ISR Incl. interrupts/blocking			
Tasks and ISR with Scheduling			
Latency Times			
Interrupt locking times			
Network Cycle Times			
Cyclic activities			

Typical situation in industry

- Performance and timing measurement is not done for most projects because of lack resources and/or know-how
- If Performance and timing measurement is done for a project, the approach and scope depends heavily on the skills and experience of the developer of the measurement solution

Wrap-Up

- Realtime criteria
- A realtime operating system ensures, that certain activities (i.e. tasks) start execution within a defined timeframe.
- It cannot guarantee however, that the execution of the activity is finished within a defined timeframe (although it may throw an exception).
- Knowing and considering timing constraints is of fundamental importance when designing a realtime system!
- Execution time can be measured dynamically
- Reliable measurement is pretty expensive!